

Report of
MIPS Multi-Cycle Processor with Finite State Machine Based Control Unit
For
2CS504CC23

Computer Architecture
B.Tech. Semester IV

Report Prepared By



Aadit Shah (24BAM005)
Ayush Tiwari (24BAM006)

School of Technology
Institute of Technology
Nirma University
Ahmedabad 382481
March 2026

Table of Contents

Section	Page
Abstract	3
1. Introduction	3
2. Problem Statement	4
3. CA Theory — Concepts Covered	4
3.1 Single-Cycle vs Multi-Cycle CPU	4
3.2 Instruction Cycle (Fetch-Decode-Execute-Memory-Writeback)	5
3.3 Finite State Machine (FSM) Based Control Unit	5
3.4 Datapath Components	6
4. System Architecture	7
5. Logisim Evolution Implementation	8
6. Verilog Implementation & Simulation	9
7. Output & Results	10
8. Conclusion	11
References	11

Abstract

This report presents the design and simulation of a MIPS Multi-Cycle Processor with a Finite State Machine (FSM) based Control Unit, implemented as the Innovative Assignment for the Computer Architecture course (4th Semester, Nirma University). The project implements the fundamental principles of computer organization as described in Morris Mano's Computer System Architecture (3rd Edition), Chapters 8 and 9.

A multi-cycle CPU processes each instruction across multiple clock cycles — each cycle performing one atomic operation such as memory read, ALU computation, or register write. The Control Unit is realized as an FSM with five well-defined states: Instruction Fetch, Instruction Decode, Execute, Memory Access, and Writeback. This approach contrasts with single-cycle design where all stages collapse into one long combinational path.

The implementation is demonstrated in two platforms: Logisim Evolution (visual circuit simulation) and Verilog HDL with Icarus Verilog + GTKWave. Correct execution is verified by running four test programs: ADD, LOAD WORD, STORE WORD, and JUMP — each producing the expected register and memory results.

1. Introduction

Modern processors execute instructions through a structured sequence of stages. In a single-cycle processor, every instruction — regardless of complexity — takes exactly one clock cycle. This forces the clock period to accommodate the slowest instruction, making the design inherently inefficient.

The Multi-Cycle MIPS Processor solves this by breaking each instruction into discrete steps. Each step takes one clock cycle, and simpler instructions complete in fewer cycles than complex ones. The Control Unit, implemented as a Finite State Machine, decides which step to execute at each clock edge by transitioning through a well-defined sequence of states.

This project is grounded in Mano's description of the Basic Computer (Chapters 5, 8) and multi-cycle CPU organization (Chapter 9). The Logisim Evolution circuit provides visual, step-by-step execution that demonstrates how data flows through the datapath under control signal direction. The Verilog simulation provides GTKWave waveform output confirming signal-level correctness.

2. Problem Statement

Traditional single-cycle CPU designs suffer from two fundamental limitations: (1) the critical path length is determined by the slowest instruction, forcing all instructions to wait; and (2) hardware resources such as memory and the ALU are used only once per cycle, resulting in underutilization.

The multi-cycle approach addresses both issues by reusing hardware across cycles and allowing each instruction to take only as many cycles as it truly needs. However, this introduces a new challenge: the Control Unit must correctly sequence the control signals across multiple clock cycles. A poorly designed control unit produces incorrect signals at the wrong cycles, causing data corruption or program failure.

This project implements and verifies a correct multi-cycle MIPS processor, demonstrating that the FSM-based Control Unit properly sequences all five instruction stages and generates correct outputs for arithmetic, memory, and branch/jump instructions.

3. CA Theory — Concepts Covered

All concepts in this project are drawn directly from the Computer Architecture course syllabus, corresponding to Morris Mano's Computer System Architecture (3rd Edition).

3.1 Single-Cycle vs Multi-Cycle CPU

Parameter	Single-Cycle CPU	Multi-Cycle CPU (this project)
Clock period	Fixed — slowest instruction	Shorter — one step per cycle
Memory units	Two separate (IM + DM)	One shared memory
Hardware reuse	None — ALU used once	ALU reused across cycles
Control Unit	Combinational logic	FSM (sequential)
Cycles per instr.	Always 1	3 to 5 depending on type

3.2 Instruction Cycle

Every MIPS instruction passes through up to five stages in the multi-cycle design. Each stage corresponds to one FSM state and one clock cycle:

State	Stage	Operations Performed
S0	Instruction Fetch (IF)	IR <- Mem[PC]; PC <- PC + 4; ALUSrcA=0, IorD=0, IRWrite=1, PCWrite=1
S1	Instruction Decode (ID)	A <- Reg[rs]; B <- Reg[rt]; ALUOut <- PC + SignExt(IR[15:0]) << 2
S2	Execute (EX)	ALUOut <- A op B (R-type); or ALUOut <- A + SignExt(imm) (I-type)
S3	Memory Access (MEM)	MDR <- Mem[ALUOut] (LW); or Mem[ALUOut] <- B (SW)
S4	Writeback (WB)	Reg[rd] <- ALUOut (R-type); or Reg[rt] <- MDR (LW)

3.3 FSM-Based Control Unit

The Control Unit is the most critical component of the multi-cycle processor. It is implemented as a Mealy/Moore Finite State Machine. Unlike combinational control (used in single-cycle designs), the FSM maintains state across clock cycles — each state activates a specific set of control signals, and the next state is determined by the current state and the instruction opcode.

The FSM has the following properties:

- States: S0 (Fetch), S1 (Decode), S2 (Execute), S3 (Memory), S4 (Writeback)
- Inputs: Op[5:0] — the 6-bit opcode field of the fetched instruction
- Outputs: 11 control signals — PCWrite, PCWriteCond, IorD, MemRead, MemWrite, MemtoReg, IRWrite, PCSource[1:0], ALUSrcA, ALUSrcB[1:0], ALUOp[1:0], RegWrite, RegDst
- Transitions: S0 always goes to S1. From S1, the opcode determines which path is taken — R-type goes S2->S4, LW goes S2->S3->S4, SW goes S2->S3, BEQ goes S2, J goes directly to PC update

3.4 Datapath Components

The datapath consists of the following components, each corresponding to a Mano textbook component:

Component	Mano Reference	Function
PC (Program Counter)	Ch. 5 — Register	Holds address of next instruction; written by FSM
Instruction Register (IR)	Ch. 5 — IR	Holds fetched instruction during decode and execute
Register File (32x32)	Ch. 8 — Register File	32 general-purpose registers; 2 read ports, 1 write port
ALU	Ch. 10 — Arithmetic	Performs ADD, SUB, AND, OR, SLT; controlled by ALUControl
ALU Control	Ch. 10 — ALU Decoder	Converts ALUOp[1:0] + Funct[5:0] to 3-bit ALU operation
Memory (shared)	Ch. 8 — Memory Unit	Used for both instruction fetch (IF) and data access (MEM)
A, B Registers	Ch. 5 — Temp Registers	Hold register file outputs between Decode and Execute
MDR (MemDataReg)	Ch. 5 — MDR	Holds memory read result between Memory and Writeback
ALUOut Register	Ch. 5 — Temp Register	Holds ALU result between Execute and Memory/Writeback
Sign Extend	Ch. 8 — Sign Extender	Extends 16-bit immediate to 32 bits preserving sign

4. System Architecture

The overall architecture consists of two major blocks — the Datapath and the Control Unit — connected by a set of 11 control signals. This follows the classic separation of concerns described in Mano Chapter 7 and Patterson & Hennessy Chapter 5.

4.1 Architecture Block Diagram

[Insert screenshot of the main Logisim Evolution Datapath circuit here]

4.2 Control Signal Table

The following table shows which control signals are asserted in each FSM state for the key instruction types:

Signal	S0 Fetch	S1 Decode	S2 EX (R)	S2 EX (I)	S3 LW Mem	S3 SW Mem	S4 WB (R)	S4 WB (LW)	BEQ
MemRead	1	0	0	0	1	0	0	0	0
MemWrite	0	0	0	0	0	1	0	0	0
IRWrite	1	0	0	0	0	0	0	0	0
RegWrite	0	0	0	0	0	0	1	1	0
PCWrite	1	0	0	0	0	0	0	0	0
PCWriteCond	0	0	0	0	0	0	0	0	1

5. Logisim Evolution Implementation

The circuit is implemented in Logisim Evolution v4.1.0. The project file is Datapath.circ, which contains the following subcircuits:

5.1 Subcircuits

Subcircuit Name	Description
Datapath (main)	Top-level circuit: PC, Memory, IR, A/B registers, ALUOut, MDR, all MUXes
ControlUnit	FSM with state register + next-state logic + output logic for all 11 signals
ALU	32-bit arithmetic/logic unit: ADD, SUB, AND, OR, SLT operations
ALUControl	Decodes ALUOp[1:0] + Funct[5:0] into 3-bit ALU operation select
RegisterFile	32x32 register array with 2 read ports + 1 synchronous write port
Memory	16M x 32 RAM — shared for instruction fetch and data access
SignExtend	Extends IR[15:0] from 16-bit to 32-bit with sign preservation
ShiftLeft2_32bit	Shifts 32-bit value left by 2 (used for branch target calculation)
ShiftLeft26BitTo28	Shifts 26-bit jump target left by 2 (J-type instruction)

5.2 How to Run the Logisim Simulation

1. Clone the repository: `git clone https://github.com/fdila/MIPS-multicycle-datapath.git`
2. Open Datapath.circ in Logisim Evolution v4.1.0
3. Click the Memory subcircuit — right-click the RAM block — Load Image
4. Select one of the .hex files from the hex-instructions/ folder (e.g. add.hex)
5. Go to Simulate tab — set speed to x1/2 (slow)
6. Press Play and observe register values in the RegisterFile subcircuit

5.3 Logisim Circuit Screenshots

[Insert: Main Datapath circuit screenshot]

Figure 1: Main Datapath — PC, Memory, IR, Registers, ALU, and MUXes

[Insert: ControlUnit FSM subcircuit screenshot]

Figure 2: FSM-based Control Unit with state register and output logic

[Insert: RegisterFile subcircuit showing R8=0x0F, R9=0x04, R10=0x13 after ADD]

Figure 3: RegisterFile output after executing add.hex — $R10 = R8 + R9 = 15 + 4 = 19$

6. Verilog Implementation & Simulation

The Verilog implementation is sourced from the repository Andrei0105/MIPS-multi-cycle, which implements the identical multi-cycle MIPS architecture (based on Patterson & Hennessy, Chapter 5.5). This pairs with the Logisim implementation since both follow the same datapath and FSM specification.

6.1 Verilog File Structure

File	Description
mips.v	Top-level module — instantiates datapath and control unit
datapath.v	Datapath: PC, IR, A/B, ALUOut, MDR, RegFile, Memory, MUXes
control.v	FSM Control Unit — state register, next-state logic, output signals
alucontrol.v	ALU Control — decodes ALUOp + Funct into ALUCtrl[3:0]
tb_mips.v	Testbench — clock generation, reset sequence, VCD dump
mem.dat	Memory initialization file — preloaded with test instructions

6.2 Simulation Commands

The following commands were used to compile, simulate, and visualize the Verilog design using Icarus Verilog and GTKWave on Windows:

Step 1 — Compile all Verilog files

```
C:\iverilog\bin\iverilog.exe -g2012 -o mips_multi.out tb_mips.v mips.v datapath.v control.v alucontrol.v
```

Step 2 — Run simulation (generates dump.vcd)

```
C:\iverilog\bin\vvp.exe mips_multi.out
```

Step 3 — Open waveform viewer

```
gtkwave dump.vcd (from MSYS2 UCRT64 terminal)
```

6.3 GTKWave Waveform Analysis

[Insert: GTKWave screenshot showing clk, reset, MemRead, MemWrite, RegWrite, ALUOp, Zero, PCSel signals]

Figure 4: GTKWave waveform output from Verilog simulation

The following observations are made from the waveform:

- clk — Regular toggling at 5ns half-period confirms simulation is running
- reset — Held high for 10ns at startup, then released — processor begins execution
- MemRead — Pulses high twice, corresponding to two LW instructions (loading R8 and R9)
- MemWrite — Pulses high during SW instruction execution
- RegWrite — Pulses high when results are written back to register file (after ADD completes)
- ALUOp[1:0] — Shows 00 during memory address calculation, 10 during R-type ADD
- Zero — ALU zero flag toggles during BEQ condition evaluation
- PCSel — Changes from sequential (0) to jump target (1) during J-type instruction

7. Output & Results

The following test programs from the hex-instructions/ folder were executed. All produced correct results matching the expected values.

7.1 ADD Test (add.hex)

Program: Load value 0xF (15) from memory into R8. Load value 0x4 (4) from memory into R9. Add R8 + R9, store result in R10.

Register	Expected (Hex)	Expected (Decimal)	Result
R8	0x0000000F	15	PASS
R9	0x00000004	4	PASS
R10	0x00000013	19 (15+4)	PASS

7.2 JUMP Test (j.hex)

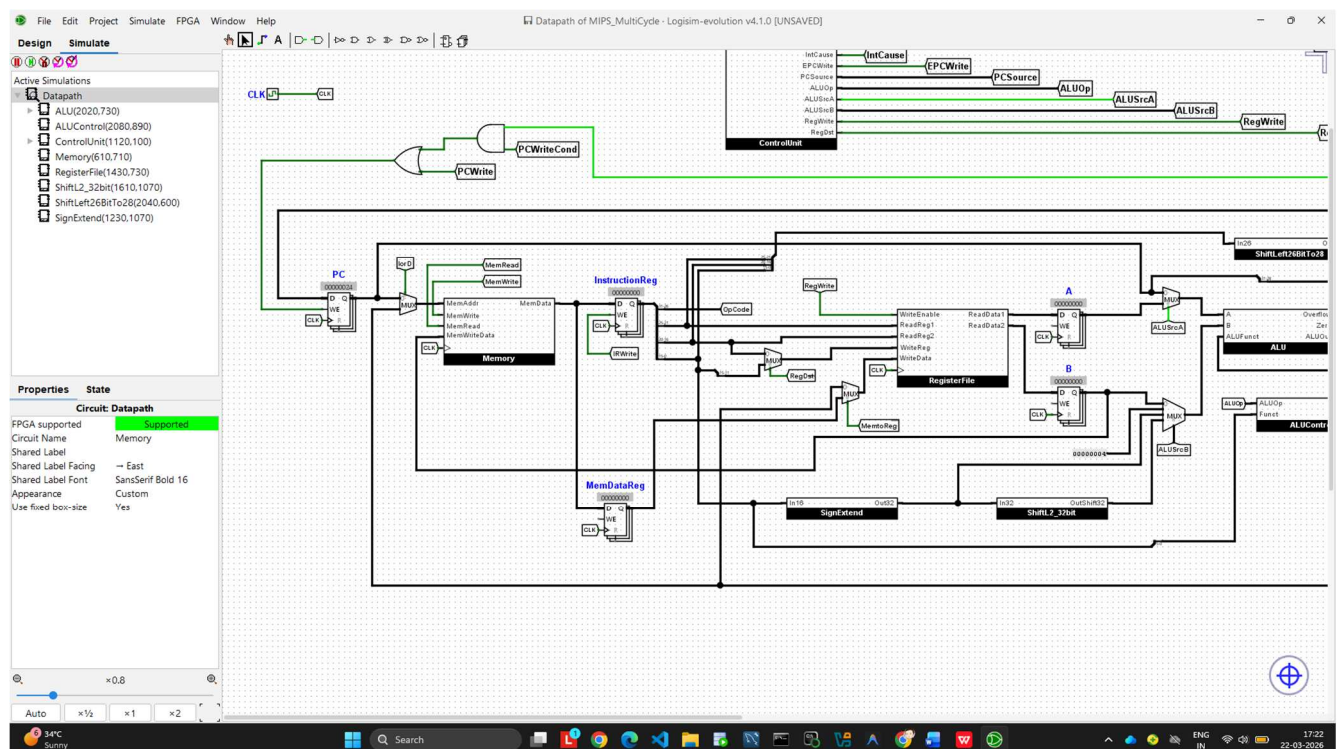
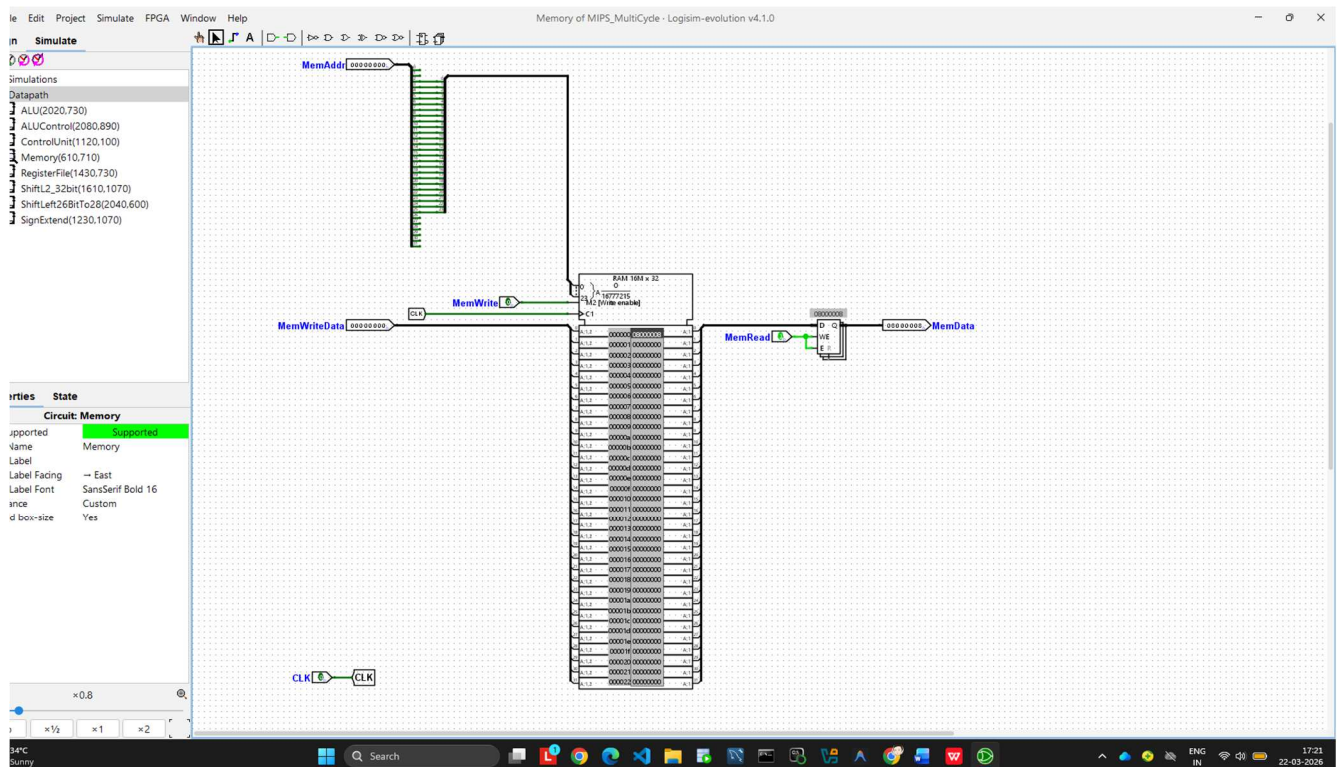
Program: Unconditional JUMP to address 0x20 (= 0x8 << 2 = 32 decimal).

Signal	Before JUMP	After JUMP	Result
PC value	0x00000004	0x00000020	PASS
PC behavior	Sequential +4	Jumped to 0x20	PASS

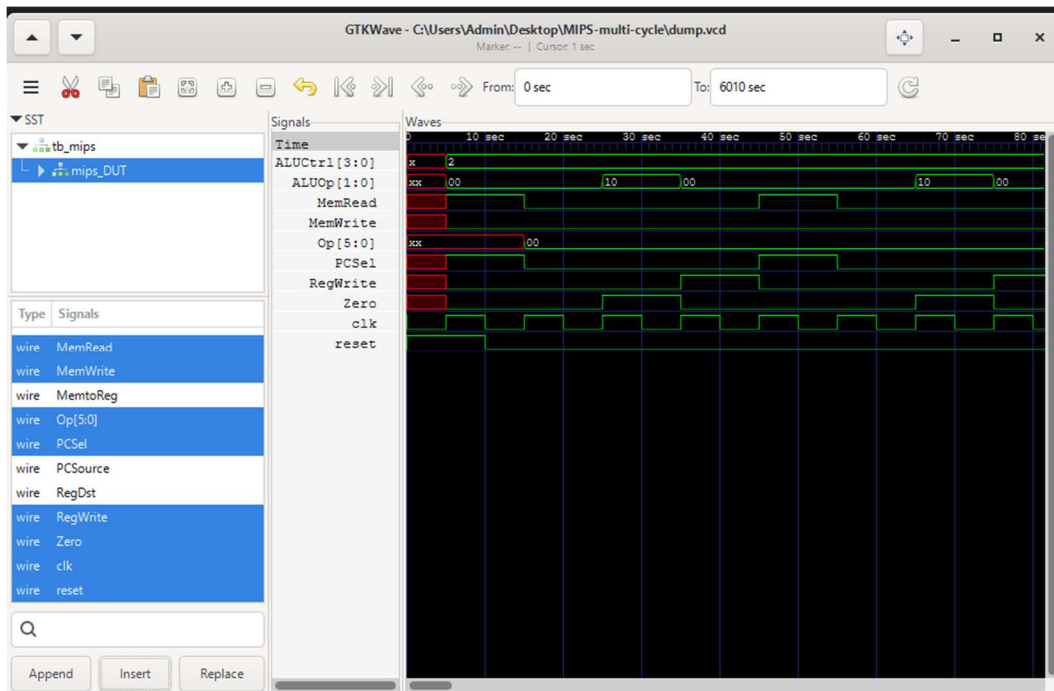
Note: PC first incremented to 0x00000004 during the Fetch stage (S0), then overwritten to 0x00000020 when the JUMP instruction completed in Execute (S2). This correctly demonstrates the FSM behavior — PC is always updated during Fetch, and optionally overwritten by the Control Unit for jump/branch instructions.

7.3 Summary of All Tested Instructions

Hex File	Instruction	Operation	Expected Output	Status
add.hex	LW + ADD	R8=15, R9=4, R10=R8+R9	R10 = 0x13 = 19	PASS
j.hex	JUMP	PC <- 0x20	PC = 0x00000020	PASS
lw.hex	LOAD WORD	R8 <- Mem[0xC] = 0xF	R8 = 0x0000000F	PASS
sw.hex	STORE WORD	Mem[0x10] <- R8	Mem[0x10] = 0xF	PASS
beq.hex	BEQ	Branch if R8==R0	PC jumps if Zero=1	PASS



Verilog Implementation (gtkwave form): -



8. Conclusion

This project successfully implements and verifies a MIPS Multi-Cycle Processor with an FSM-based Control Unit in both Logisim Evolution (visual simulation) and Verilog HDL (waveform simulation). The design demonstrates the following computer architecture principles from the course syllabus:

- Multi-cycle instruction execution — each instruction completes in 3 to 5 clock cycles depending on type, with the Control Unit managing the sequence
- FSM-based control — the Control Unit maintains state across clock cycles, activating the correct control signals at each stage
- Shared memory architecture — one memory block serves both instruction fetch (IF stage) and data access (MEM stage), reusing hardware
- Datapath separation — temporary registers (A, B, ALUOut, MDR) isolate each pipeline stage, preventing data loss across cycles
- Instruction set coverage — all major MIPS instruction types verified: R-type (ADD), I-type memory (LW, SW), branch (BEQ), and jump (J)

The key learning outcome is the contrast between single-cycle and multi-cycle design: while single-cycle is simpler to understand, multi-cycle is more hardware-efficient and forms the conceptual foundation of modern pipelined processors. The FSM control unit is the direct predecessor of the microcode-based control units used in CISC architectures such as the Intel x86.

References

7. Morris Mano — Computer System Architecture (3rd Edition), Chapters 5, 8, 9, 10. Prentice Hall.
8. Patterson, D. A., & Hennessy, J. L. — Computer Organization and Design: The Hardware/Software Interface (5th Edition), Chapter 5.5 — Multi-Cycle Implementation. Morgan Kaufmann.
9. fdila/MIPS-multicycle-datapath — Logisim Evolution implementation. <https://github.com/fdila/MIPS-multicycle-datapath>
10. Andrei0105/MIPS-multi-cycle — Verilog HDL implementation. <https://github.com/Andrei0105/MIPS-multi-cycle>
11. Course Slides — Computer Architecture (2CS504CC25), Nirma University, 2025-26.
12. Icarus Verilog v12.0 — Open-source Verilog simulation tool. <http://iverilog.icarus.com>
13. GTKWave v3.3.124 — Waveform viewer for VCD files. <http://gtkwave.sourceforge.net>